

The Synthesis initial Concept

<https://claude.ai/chat/979ce334-4514-4b37-a882-0df89d1cca0d>

Here's my completed analysis, improvement recommendations, and a demo prototype:

****Design evaluation****

*****Potential:**** The concept of a self-extending agentic AI is compelling. Dynamically generating new MCP servers from high-level needs and sharing them through a repository could yield a rich ecosystem of reusable tools. The proposed trust-scoring and sandboxing mechanisms align with best practices for mitigating risk.

*****Plausibility:**** While dynamic tool creation is feasible, the document overstates the maturity of automatic code synthesis. Recent research shows that LLM-generated code often contains logic errors and requires rigorous tests and manual fixes. A practical system should emphasise measurement and gradual roll-out rather than claiming high success rates.

*****Correct aspects:****

1. The MCP server factory shows how to generate and register new tools at runtime.
2. Trust-level gating using execution statistics and sandbox configurations recognises the need for graduated permissions.
3. A shared repository with ratings and forking encourages community reuse and incremental improvement.

*****Flaws:****

1. The document asserts an "85 %+ success rate" for TDD-driven synthesis without evidence; studies show LLMs still produce many errors.
2. The server code template naively injects user-supplied implementation code, risking syntax errors and security vulnerabilities.
3. Sandboxing and dependency management are incomplete; isolating untrusted code properly requires more than limiting built-ins and running pip installs.

****Comprehensive report & demonstration****

A detailed report outlining these points, proposing concrete improvements (such as safer code injection, realistic success measurement, and restricted execution environments), and illustrating a minimal capability manager with a Gradio interface is provided in the example file, source code is at the end of this message.

Analysis and Improvement Report

Overview of the uploaded design and code

The uploaded document is a mix of Q&A dialogue and a large amount of code and diagrammatic descriptions. It outlines an ambitious self-extending AI framework called Synthesis. The core idea is that a local agentic AI (for example the user's agent "Eve") should be able to create new capabilities on the fly when it encounters a task it cannot perform. The document proposes two complementary pieces:

MCP server factory. A Python `MCPServerFactory` class can programmatically generate MCP servers (Anthropic's message-based tool interface) by taking a `MCPServerDefinition` dataclass, writing out a Python file containing the server implementation, installing any required third-party packages via pip, and registering the new tool in Claude Desktop's configuration. An `AgenticAIToolManager` class wraps this factory so that an agent can call `need_tool` to create a new tool.

Synthesis system. The second, much larger section describes a proposed architecture for automatically synthesising capabilities using test-driven development (TDD). It includes dataclasses for representing capabilities and their metadata, a repository for sharing and rating capabilities, a trust-scoring system that promotes capabilities from UNTRUSTED to VERIFIED as they pass more executions, and sandboxing classes that run untrusted code in Docker or a separate process. There is also mention of an `EvolutionEngine` that watches usage metrics and automatically generates improved versions of capabilities.

The document's intent is visionary: a self-improving ecosystem where AI agents build, share and evolve their own tools, governed by test-driven validation and trust metrics.

Potential and plausibility

Potential: The high-level vision has significant appeal. Decoupling tool creation from any particular assistant enables agents to extend themselves, while a shared repository with ratings and automatic evolution could create a “network effect” where successful solutions are reused and improved. Combining TDD with long-running agents addresses a real weakness of large language models (LLMs): they often generate code that is syntactically correct but logically wrong. Recent research shows that incorporating tests during code generation can improve correctness, and iterative feedback loops allow LLMs to refine their outputs

arxiv.org

.

Plausibility: Some aspects are plausible, but others are overly optimistic or incomplete. Running locally created MCP servers and adding them to an agent’s tool list is straightforward. Maintaining metadata and trust levels is likewise feasible with databases and file storage. However, the claim that this framework would achieve an “85%+ success rate” in automatic synthesis and that untrusted code can be safely sandboxed simply by counting executions is not supported by evidence. Empirical studies demonstrate that current LLMs still struggle to reliably generate correct code and make a wide range of semantic and syntactic errors

arxiv.org

. Effective TDD requires high-quality human-written tests; automatically generated tests are often as flawed as the generated code

arxiv.org

. The design glosses over complexities such as secure code execution, dependency management, and how an agent would specify its needs precisely enough for automatic synthesis.

Three things the design gets right

Dynamic tool generation. The MCPServerFactory demonstrates how an agent can create new tools at runtime by assembling Python code, writing it to disk and exposing it via the MCP protocol. This dynamic approach opens the door for agents to augment their capabilities without waiting for human developers.

Trust-level gating. The proposal recognises that unvetted code should run in restricted environments and gradually earn more privileges. The TrustScore and SandboxConfig classes define escalating isolation levels and criteria for promotion. This reflects best practices in secure execution by limiting resource usage, network access and subprocess spawning based on a capability’s track record.

Shared repository concept. A central CapabilityRepository allows capabilities to be published, downloaded, rated and forked. A vibrant marketplace of reusable tools could indeed produce network effects: more agents contribute more capabilities, which increases the value of the repository for everyone.

Three things the design gets wrong

Overstated success claims. The document repeatedly asserts that the TDD synthesiser achieves an “85%+ success rate” and that generating tests first virtually ensures correct code. Recent empirical studies show that LLMs still fail frequently when generating code, and many errors require substantial manual fixing

arxiv.org

. Tests generated by LLMs can miss edge cases or mis-specify requirements

arxiv.org

. Presenting high success rates without evidence misleads users about the reliability of such a system.

Unsafe code generation patterns. The generate_server_code method injects arbitrary implementation_code directly into the server template without any sanitisation or indentation control. If the supplied code contains mismatched indentation or malicious constructs, the resulting file will not run correctly or could execute unintended actions. Similarly, installing arbitrary packages via pip at runtime without constraints can lead to dependency conflicts or security risks.

Incomplete sandboxing and dependency handling. The proposal mentions Docker- and process-level sandboxes but does not implement them fully; for example, DockerSandbox references an imported docker module that is never imported. More importantly, isolating untrusted Python code is non-trivial. Running arbitrary user-supplied code in the same process with restricted __builtins__ is not enough to guarantee safety. The design also assumes that dependencies listed in requirements can be installed on demand, but does not consider version conflicts, offline environments or malicious packages.

Proposed improvements

To make the system more robust and realistic, several changes are necessary:

Ground expectations in current research. The design should acknowledge that LLM-generated code often contains logical errors. Research shows that LLMs produce compilable but incorrect code, and high-quality tests are required to catch these errors [arxiv.org](#)

. Test-driven synthesis can help, but its success rate depends on the difficulty of the task, the quality of the tests and the model used [arxiv.org](#)

. Claims about achieving a specific success rate should be replaced with a process for measuring and reporting real-world results.

Improve code generation hygiene. Instead of concatenating user-supplied code into a template, the factory should treat the implementation as a separate module or callable. For example, it could write the implementation to a separate file and import it, or wrap it in a well-defined function signature. Any indentation issues should be normalised. The factory should also avoid globally installing packages; instead, it could run in isolated virtual environments, and only allow installation of vetted packages.

Implement safer execution of dynamic code. True sandboxing is difficult in pure Python. Until Docker-based isolation is fully implemented, the system can mitigate risk by limiting built-ins and preventing dangerous modules from being imported. For local prototyping, a restricted exec environment with a curated set of safe functions (e.g., `abs`, `len`, `range`) can reduce risk. For production, untrusted code should run in a separate process or container with explicit resource limits.

Simplify the MVP. The initial version should focus on a narrow scope: let users define simple functions, provide tests and run them in a restricted environment. Trust scoring can be based on the number of successful runs and test pass rate. Once this core loop works reliably, additional features such as repositories, ratings and automatic evolution can be layered on.

Implementation: a simplified capability manager and demo

The following implementation demonstrates a minimal viable product inspired by the uploaded design. It implements a `CapabilityManager` that lets users dynamically add Python functions (capabilities), validate them with tests, run them safely with restricted built-ins and track a simple trust score. A small Gradio interface allows the user to create new capabilities, run them and view their trust level. This demo illustrates the core principles of dynamic tool creation, test-driven validation and trust-level promotion.

Key design choices

Restricted execution: User-provided code is executed with a limited set of built-ins to reduce the risk of malicious operations. Only safe functions such as `abs`, `min`, `max`, `sum`, `len`, `range` and `math` are exposed. Filesystem and network access are not available.

Function extraction: The system expects the user code to define exactly one function. The first callable defined in the executed code is considered the capability function. Errors during compilation are reported back to the user.

Test specification: Tests are provided as a JSON list of objects with input and output fields. The input field must be a list of positional arguments; the output field is the expected return value. The manager runs each test, compares the result using equality and updates the capability's success metrics.

Simple trust scoring: A capability starts as `UNTRUSTED`. If it has at least 10 executions with a success rate of 90% or higher it becomes `PROBATION`; after 50 executions with a success rate of 95% or higher it becomes `TRUSTED`. (The `VERIFIED` level from the original design, which requires human verification, is omitted in this demo.)

Gradio UI: The interface provides two tabs: one to create a capability and run its tests, and another to execute an existing capability on arbitrary inputs and see the updated trust level.

Code listing

The following Python file implements the simplified capability manager and Gradio application. Save it as `gradio_demo.py` and run it with `python gradio_demo.py` to launch the interface.

```
import json
import math
from dataclasses import dataclass
from enum import Enum
from typing import Any, Callable, Dict, List, Tuple
```

```
import gradio as gr
```

```
class TrustLevel(Enum):
    """Simple trust levels used in the demo."""
    UNTRUSTED = "UNTRUSTED"
    PROBATION = "PROBATION"
    TRUSTED = "TRUSTED"
```

```
@dataclass
class Capability:
    """A dynamically created function with a trust score."""
    name: str
    description: str
    code: str
    func: Callable
    total_executions: int = 0
    successful_executions: int = 0

    @property
    def success_rate(self) -> float:
        if self.total_executions == 0:
            return 0.0
        return self.successful_executions / self.total_executions
```

```
    @property
    def trust_level(self) -> TrustLevel:
        """Compute the trust level based on executions and success rate."""
        if self.total_executions >= 50 and self.success_rate >= 0.95:
            return TrustLevel.TRUSTED
        if self.total_executions >= 10 and self.success_rate >= 0.90:
            return TrustLevel.PROBATION
        return TrustLevel.UNTRUSTED
```

```
class CapabilityManager:
    """Manages dynamic capabilities and their trust scores."""

    def __init__(self) -> None:
        self.capabilities: Dict[str, Capability] = {}

    @staticmethod
    def _safe_exec(user_code: str) -> Tuple[bool, Any, str]:
        """Execute user code in a restricted environment and extract the first function.

        Returns a tuple `(success, function, message)`. If `success` is False,
        `message` contains an error description; otherwise `function` is the extracted callable.
```

```
"""
```

```
# Define a minimal set of safe builtins and modules
```

```
allowed_builtins = {
```

```
    'abs': abs,
```

```
    'min': min,
```

```
    'max': max,
```

```
    'sum': sum,
```

```
    'len': len,
```

```
    'range': range,
```

```
    'sorted': sorted,
```

```
    'math': math,
```

```
}
```

```
namespace: Dict[str, Any] = {}
```

```
try:
```

```
    exec(user_code, {'__builtins__': allowed_builtins}, namespace)
```

```
except Exception as e:
```

```
    return False, None, f"Compilation error: {e}"
```

```
# Find the first callable defined in namespace
```

```
func = None
```

```
for obj in namespace.values():
```

```
    if callable(obj):
```

```
        func = obj
```

```
        break
```

```
if func is None:
```

```
    return False, None, "No function definition found in the provided code."
```

```
return True, func, ""
```

```
def add_capability(self, name: str, description: str, code: str, tests_json: str) -> str:
```

```
    """Add a new capability and run its tests. Returns a result string."""
```

```
    success, func, message = self._safe_exec(code)
```

```
    if not success:
```

```
        return message
```

```
    capability = Capability(name=name, description=description, code=code, func=func)
```

```
    # Run tests if provided
```

```
    if tests_json.strip():
```

```
        try:
```

```
            tests = json.loads(tests_json)
```

```
        except json.JSONDecodeError as e:
```

```
            return f"Invalid JSON for tests: {e}"
```

```
        if not isinstance(tests, list):
```

```
            return "Tests JSON must be a list of objects with 'input' and 'output'."
```

```
        total = 0
```

```
        passed = 0
```

```
        for t in tests:
```

```
            if not isinstance(t, dict) or 'input' not in t or 'output' not in t:
```

```
                return "Each test must be an object with 'input' and 'output' keys."
```

```
            args = t['input']
```

```
            expected = t['output']
```

```
            if not isinstance(args, list):
```

```
                return "The 'input' field must be a list of positional arguments."
```

```
            total += 1
```

```
            try:
```

```
                result = func(*args)
```

```
                if result == expected:
```

```

        passed += 1
        capability.successful_executions += 1
        capability.total_executions += 1
    except Exception:
        capability.total_executions += 1
    result_msg = f"Tests passed: {passed}/{total}. Trust level: {capability.trust_level.value}"
else:
    result_msg = "Capability added without tests. Trust level: UNTRUSTED"
self.capabilities[name] = capability
return result_msg

def run_capability(self, name: str, args_json: str) -> str:
    """Execute an existing capability with arguments provided as JSON list."""
    if name not in self.capabilities:
        return f"No capability named '{name}'."
    capability = self.capabilities[name]
    try:
        args = json.loads(args_json) if args_json.strip() else []
        if not isinstance(args, list):
            return "Arguments must be provided as a JSON list."
    except json.JSONDecodeError as e:
        return f"Invalid JSON for arguments: {e}"
    capability.total_executions += 1
    try:
        result = capability.func(*args)
        capability.successful_executions += 1
        return f"Result: {result}\nSuccess rate: {capability.success_rate:.2f}\nTrust level: {capability.trust_level.value}"
    except Exception as e:
        return f"Execution failed: {e}\nSuccess rate: {capability.success_rate:.2f}\nTrust level: {capability.trust_level.value}"

```

```

# Create a single CapabilityManager instance
manager = CapabilityManager()

```

```

def ui_add_capability(name: str, description: str, code: str, tests: str) -> str:
    return manager.add_capability(name.strip(), description.strip(), code, tests)

```

```

def ui_run_capability(name: str, args: str) -> str:
    return manager.run_capability(name.strip(), args)

```

```

def get_capability_names() -> List[str]:
    return list(manager.capabilities.keys())

```

```

with gr.Blocks() as demo:
    gr.Markdown("""# Dynamic Capability Manager Demo

```

This demo lets you define Python functions on the fly, test them and then run them.

Provide the function definition in the ****Code**** box. It must define at least one function (any name is accepted; the first defined function will be used).

Specify tests as a JSON list of objects with `"input"` (a list of arguments) and `"output"` (the expected return value). When you run the tests, the system records successes and failures and computes a trust level.

```
"""
with gr.Tab("Create Capability"):
    cap_name = gr.Textbox(label="Capability Name", placeholder="e.g. square")
    cap_desc = gr.Textbox(label="Description", placeholder="Describe what the function does")
    cap_code = gr.Textbox(label="Code", placeholder="def square(x):\n    return x*x", lines=10)
    cap_tests = gr.Textbox(label="Tests (JSON)", placeholder='[\n {"input": [3], "output": 9},\n {"input": [5], "output": 25}\n] ',
lines=6)
    add_button = gr.Button("Add Capability & Run Tests")
    add_output = gr.Textbox(label="Result", interactive=False)
    add_button.click(ui_add_capability, inputs=[cap_name, cap_desc, cap_code, cap_tests], outputs=add_output)
with gr.Tab("Run Capability"):
    sel_cap = gr.Dropdown(label="Select Capability", choices=lambda: get_capability_names())
    run_args = gr.Textbox(label="Arguments (JSON list)", placeholder="[10]")
    run_button = gr.Button("Run Capability")
    run_output = gr.Textbox(label="Execution Result", interactive=False)
    run_button.click(ui_run_capability, inputs=[sel_cap, run_args], outputs=run_output)

if __name__ == "__main__":
    demo.launch()
```

Running the demo

Install the dependencies (Gradio) if not already installed:

```
pip install gradio
```

Save the code above as `gradio_demo.py` and run it:

```
python gradio_demo.py
```

Open the URL printed by Gradio (typically `http://127.0.0.1:7860`) in a browser. Use the Create Capability tab to define a function (for example,